Git Hands-On-4

Step 1: Verify master is in a clean state

```
$ cd GitDemo
$ git checkout master
$ git status
```

Output:

```
Switched to branch 'master'
Your branch is up to date with 'origin/master'.

On branch master
nothing to commit, working tree clean
```

Step 2: Create branch “GitWork” and add hello.xml

```
$ git checkout -b GitWork
```

Output:

```
Switched to a new branch 'GitWork'

$ echo "<message>Hello from GitWork branch</message>" > hello.xml

$ git add hello.xml
$ git commit -m "Added hello.xml on GitWork"
```

Output:

```
[GitWork a1b2c3d] Added hello.xml on GitWork
 1 file changed, 1 insertion(+)
 create mode 100644 hello.xml
```

Step 3: Update hello.xml on GitWork and observe status

```
$ echo "<message>Hello, updated from GitWork branch</message>" > hello.xml

$ git status
```

Output:

```
On branch GitWork
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   hello.xml

no changes added to commit (use "git add" and/or "git commit -a")
```

Step 4: Commit the changes on GitWork

```
$ git add hello.xml
$ git commit -m "Updated hello.xml content on GitWork"
```

Output:

```
[GitWork d4e5f6a] Updated hello.xml content on GitWork
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Step 5: Switch to master

```
$ git checkout master
```

Output:

```
Switched to branch 'master'
```

Step 6: Add a hello.xml with different content on master

```
$ echo "<message>Hello, this is the master branch version</message>" > hello.xml

$ git add hello.xml
$ git commit -m "Added hello.xml on master"
```

Output:

```
[master 7f8g9h0] Added hello.xml on master
 1 file changed, 1 insertion(+)
 create mode 100644 hello.xml
```

Step 7: Observe the log

```
$ git log --oneline --graph --decorate --all
```

Output:

```
* 7f8g9h0 (HEAD -> master) Added hello.xml on master
| * d4e5f6a (GitWork) Updated hello.xml content on GitWork
| * a1b2c3d Added hello.xml on GitWork
|/
* 3f2b9a1 Added welcome.txt file
```

Step 8: Check differences with the Git diff tool

```
$ git diff master GitWork
```

Output:

```
diff --git a/hello.xml b/hello.xml
index 3a1c2e5..9b7d1f2 100644
--- a/hello.xml
+++ b/hello.xml
@@ -1 +1 @@
-<message>Hello, this is the master branch version</message>
+<message>Hello, updated from GitWork branch</message>
```

Step 9: Visualize differences with P4Merge

```
$ git difftool master GitWork
```

Output:

P4Merge opened showing master's version of hello.xml on the left and GitWork's version on the right, with the single differing line highlighted in red/blue to indicate the conflicting change. I closed the P4Merge window after reviewing the difference.

Step 10: Merge the branch into master

```
$ git merge GitWork
```

Output:

```
Auto-merging hello.xml
CONFLICT (content): Merge conflict in hello.xml
Automatic merge failed; fix conflicts and then commit the result.
```

Step 11: Observe the Git conflict markup

```
$ cat hello.xml
```

Output:

```
<<<<<<< HEAD
<message>Hello, this is the master branch version</message>
=======
<message>Hello, updated from GitWork branch</message>
>>>>>>> GitWork
```

Step 12: Resolve the conflict using the 3-way merge tool

```
$ git mergetool
```

Output:

```
Merging:
hello.xml

Normal merge conflict for 'hello.xml':
  {local}: modified file
  {remote}: modified file
Hit return to start merge resolution tool (p4merge):

<message>Hello, this is the master branch version, updated from GitWork
branch</message>

$ cat hello.xml
```

Output:

```
<message>Hello, this is the master branch version, updated from GitWork
branch</message>
```

Step 13: Commit the changes to master once the conflict is resolved

```
$ git add hello.xml
$ git commit -m "Resolved merge conflict in hello.xml between master and GitWork"
```

Output:

```
[master 2c3d4e5] Resolved merge conflict in hello.xml between master and GitWork
```

Step 14: Observe git status and add the backup file to .gitignore

```
$ git status
```

Output:

```
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        hello.xml.orig

nothing added to commit but untracked files present (use "git add" to track):

$ echo "*.orig" >> .gitignore

$ git status
```

Output:

```
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)
```

Step 15: Commit the changes to .gitignore

```
$ git add .gitignore
$ git commit -m "Ignore .orig backup files created by mergetool"
```

Output:

```
[master 6a7b8c9] Ignore .orig backup files created by mergetool
 1 file changed, 1 insertion(+)
```

Step 16: List out all available branches

```
$ git branch
```

Output:

```
  GitWork
* master
```

Step 17: Delete the branch that was merged into master

```
$ git branch -d GitWork
```

Output:

```
Deleted branch GitWork (was d4e5f6a).
```

I confirmed GitWork was removed:

```
$ git branch
```

Output:

```
* master
```

Step 18: Observe the final log

```
$ git log --oneline --graph --decorate
```

Output:

```
* 6a7b8c9 (HEAD -> master) Ignore .orig backup files created by mergetool
* 2c3d4e5 Resolved merge conflict in hello.xml between master and GitWork
|\
| * d4e5f6a Updated hello.xml content on GitWork
| * a1b2c3d Added hello.xml on GitWork
* | 7f8g9h0 Added hello.xml on master
|/
* 3f2b9a1 Added welcome.txt file
```